A Generic Format Language for Bit Parsing

Notes on `code/format-lib/Format1_Seq.v`

Stan Kestens

October 2026

`code/rv32-decode/Parser.v` contains a parser and a serializer for one instruction format (RISC-V I-type), together with a roundtrip proof written specifically for that format. `Format1_Seq.v` generalises this: a format is described by a value of an inductive type, the parser and serializer are defined by recursion on that description, and the correctness properties are proven once for every format. These notes write down what has been defined and proven so far.

1 Setting

Let $\mathbb{B} = \{0, 1\}$ and let $\mathbb{B}^*$ be the set of finite bit strings (in Rocq: `list bool`). We write $u \cdot v$ for concatenation, $|u|$ for the length and ε for the empty string. A parser may fail; we write $X_\bot := X \cup \{\bot\}$ (in Rocq: `option X`).

Definition 1 (take)**.** For $n \in \mathbb{N}$, $\mathsf{take}_n : \mathbb{B}^* \to (\mathbb{B}^* \times \mathbb{B}^*)_\bot$ splits off the first n bits: $\mathsf{take}_n(b) = (u, r)$ if $b = u \cdot r$ with $|u| = n$, and $\mathsf{take}_n(b) = \bot$ if $|b| < n$.

Lemma 2. *For all n, b, u and r:*

(a) if $|u| = n$ then $\mathsf{take}_n(u \cdot r) = (u, r)$; ***take_bits_app***

(b) if $\mathsf{take}_n(b) = (u, r)$ then $b = u \cdot r$; ***take_bits_split***

(c) if $\mathsf{take}_n(b) = (u, r)$ then $|u| = n$. ***parsing_keeps_length***

Proof. (a) By induction on u. (b) and (c) by induction on n, with a case distinction on b: if $b = \varepsilon$ and $n > 0$ the parser fails, which contradicts the hypothesis. □

2 Format descriptions

Definition 3 (Formats)**.** Formats are given by the grammar

$$f \ ::= \ \mathsf{Bits}\ n \ \mid \ \mathsf{Seq}\ f_1\ f_2 \qquad (n \in \mathbb{N}),$$

where $\mathsf{Bits}\ n$ is a field of exactly n bits and $\mathsf{Seq}\ f_1\ f_2$ is f_1 followed by f_2.

Definition 4 (Interpretation)**.** Each format f has a set of values $[\![f]\!]$:

$$[\![\mathsf{Bits}\ n]\!] = \mathbb{B}^*, \qquad [\![\mathsf{Seq}\ f_1\ f_2]\!] = [\![f_1]\!] \times [\![f_2]\!].$$

In Rocq this is `denote : fmt -> Type`, a function from values to types, which is why the parser below has a dependent type.

Since $[\![\mathsf{Bits}\ n]\!]$ is all of $\mathbb{B}^*$ rather than strings of length n, the interpretation does not record field lengths. These are added as a separate predicate.

Definition 5 (Well-formedness)**.** $\mathsf{wf}_f \subseteq [\![f]\!]$ is defined by

$$\mathsf{wf}_{\mathsf{Bits}\ n}(v) \iff |v| = n, \qquad \mathsf{wf}_{\mathsf{Seq}\ f_1\ f_2}(v_1, v_2) \iff \mathsf{wf}_{f_1}(v_1) \wedge \mathsf{wf}_{f_2}(v_2).$$

Definition 6 (Serializer and parser)**.** $\mathsf{ser}_f : [\![f]\!] \to \mathbb{B}^*$ and $\mathsf{par}_f : \mathbb{B}^* \to ([\![f]\!] \times \mathbb{B}^*)_\perp$ are defined by recursion on f:

$$\begin{aligned}
\mathsf{ser}_{\mathsf{Bits}\ n}(v) &= v, \\
\mathsf{ser}_{\mathsf{Seq}\ f_1\ f_2}(v_1, v_2) &= \mathsf{ser}_{f_1}(v_1) \cdot \mathsf{ser}_{f_2}(v_2), \\
\mathsf{par}_{\mathsf{Bits}\ n}(b) &= \mathsf{take}_n(b), \\
\mathsf{par}_{\mathsf{Seq}\ f_1\ f_2}(b) &= \begin{cases} ((v_1, v_2), b_2) & \text{if } \mathsf{par}_{f_1}(b) = (v_1, b_1) \text{ and } \mathsf{par}_{f_2}(b_1) = (v_2, b_2), \\ \perp & \text{otherwise.} \end{cases}
\end{aligned}$$

Example 7 (I-type)**.** With $f_I \coloneqq \mathsf{Seq}\ (\mathsf{Bits}\ 7)\ (\mathsf{Seq}\ (\mathsf{Bits}\ 5)\ (\mathsf{Seq}\ (\mathsf{Bits}\ 3)\ (\mathsf{Seq}\ (\mathsf{Bits}\ 5)\ (\mathsf{Bits}\ 12))))$ we get the fields opcode, rd, funct3, rs1 and imm, read from bit 0 upwards, as in `Parser.v`. Here $[\![f_I]\!] = \mathbb{B}^* \times (\mathbb{B}^* \times (\mathbb{B}^* \times (\mathbb{B}^* \times \mathbb{B}^*)))$.

3 Correctness

The three properties below are stated with an arbitrary rest r, because that is what makes the induction step for Seq go through: the parser for f_1 has to stop exactly where f_2 begins.

Theorem 8 (Roundtrip)**.** *For every format f, every $v \in [\![f]\!]$ and every $r \in \mathbb{B}^*$:*

$$\mathsf{wf}_f(v) \implies \mathsf{par}_f(\mathsf{ser}_f(v) \cdot r) = (v, r).$$

parse_serialize

Proof. By induction on f, for all r. For Bits n this is Lemma 2(a). For Seq f_1 f_2 and $v = (v_1, v_2)$, reassociate $\mathsf{ser}_{f_1}(v_1) \cdot \mathsf{ser}_{f_2}(v_2) \cdot r$. The induction hypothesis for f_1 with rest $\mathsf{ser}_{f_2}(v_2) \cdot r$ gives $(v_1, \mathsf{ser}_{f_2}(v_2) \cdot r)$, and the one for f_2 with rest r gives (v_2, r). □

Theorem 9 (Exactness)**.** *For every format f and all b, v, r:*

$$\mathsf{par}_f(b) = (v, r) \implies b = \mathsf{ser}_f(v) \cdot r.$$

serialize_parse

Proof. By induction on f, for all b. For Bits n this is Lemma 2(b). For Seq f_1 f_2, a successful parse means $\mathsf{par}_{f_1}(b) = (v_1, b_1)$ and $\mathsf{par}_{f_2}(b_1) = (v_2, r)$. By the induction hypotheses $b = \mathsf{ser}_{f_1}(v_1) \cdot b_1$ and $b_1 = \mathsf{ser}_{f_2}(v_2) \cdot r$, so $b = \mathsf{ser}_{f_1}(v_1) \cdot \mathsf{ser}_{f_2}(v_2) \cdot r$. If one of the two parses fails, the hypothesis is false. □

Theorem 10 (Closure)**.** *For every format f and all b, v, r:*

$$\mathsf{par}_f(b) = (v, r) \implies \mathsf{wf}_f(v).$$

parse_wf

Proof. By induction on f. For Bits n this is Lemma 2(c); for Seq it follows directly from the induction hypotheses. □

Corollary 11 (I-type)**.** *For all $v \in [\![f_I]\!]$ with $\mathsf{wf}_{f_I}(v)$: $\mathsf{par}_{f_I}(\mathsf{ser}_{f_I}(v)) = (v, \varepsilon)$.* *itype_roundtrip*

Proof. Theorem 8 with $r = \varepsilon$. □

This is the theorem of `Parser.v`, now without a single proof step specific to I-type.